ZEROTRACE

A THREE-MODULE UNIFIED CYBER DEFENSE PLATFORM

Document Project Documentation

Version 1.0

Date August 2026

Author Aniruddh Gupta

"They leave no trace. Because we found them first."

CONTENTS

1. Executive Summary
2. The ZeroTrace Philosophy
3. Why ZeroTrace? — The Problem Landscape
4. Module Architecture: The Three Traces
5. Module 1: Recon — Attack Surface Discovery
6. Module 2: Intel — AI-Powered Vulnerability Prioritization
7. Module 3: Guard — LLM Security Middleware
8. ZeroTrace Unified: The Emergent Capabilities
9. The Operational Readiness Score
10. Research-Backed Gap-to-Solution Mapping
11. Why ZeroTrace Specifically? — Alternatives Considered
12. Hardware Fit & Deployment Requirements
13. Technology Stack — 2026
14. 14-Week Development Plan
15. References
16. Appendices

1. Executive Summary

ZeroTrace is a unified cyber defense platform combining three integrated modules — **Recon** (attack surface discovery), **Intel** (AI-powered vulnerability prioritization), and **Guard** (LLM security middleware) — into a single command center with a shared asset inventory, correlated threat intelligence, and one Operational Readiness Score.

Built for organizations that cannot afford enterprise SOC tooling, ZeroTrace runs on a single laptop via Docker Compose, requires zero GPU, and costs nothing to deploy. It is designed to address three critical gaps identified in 2026 cybersecurity research: **asset visibility blind spots**, **CVE prioritization failure**, and **unmonitored LLM attack vectors**.

> *"They leave no trace. Because we found them first."*

2. The ZeroTrace Philosophy

ZeroTrace operates on the principle that the best defense is **total visibility** — you cannot stop what you cannot see. It traces every asset, every vulnerability, and every AI attack vector back to a single source of truth.

PRINCIPLE	MEANING
Zero	Zero blind spots. Zero unpatched critical vulns. Zero AI attacks unblocked.
Trace	Every asset traced. Every threat traced. Every attack traced to its origin.

3. Why ZeroTrace? — The Problem Landscape

3.1 The Asset Visibility Crisis

Organizations cannot protect assets they do not know exist. Shadow IT — the use of IT systems without explicit organizational approval — has become a critical issue with cloud computing, introducing significant security vulnerabilities due to lack of oversight and proper security measures.

According to the **Intruder 2026 Attack Surface Management Index**, which analyzed anonymized data from 3,000 organizations:

- **54%** of SMEs (1–250 employees) had at least one attack surface exposure in the past 12 months
- **26%** had MySQL databases exposed to the internet
- **26%** exposed risky ports and services

- **20%** exposed HTTP admin panels

As organizations grow, the number of internet-facing assets increases sharply, making security management significantly more complex.

3.2 The Vulnerability Prioritization Crisis

Organizations face approximately **135 new CVEs per day** — a 40% year-on-year increase — with over **48,000 new CVEs** published in 2025 alone. Security teams following CVSS-based prioritization must address thousands of vulnerabilities that pose little real-world risk.

Research has established that **CVSS-based prioritization is highly inefficient**, with efficiency measurements (0.2–0.5%) being even lower than previous research (3.9%). The Common Vulnerability Scoring System relies on manually assigned metrics that are often incomplete or inconsistent across different reporters.

Mean Time to Remediate (MTTR) stretches to **55–72 days** for critical vulnerabilities. Research shows that vulnerabilities with high exploitation probability (EPSS > 0.7) actually carry a higher average remediation time (109 days) than low-probability ones — meaning teams are systematically patching the wrong things first.

Picus Security reports that organizations using validation-led approaches can reduce MTTR from **74 days → 14 days** and clear **86%** of patch backlogs.

3.3 The LLM Security Crisis

While nearly **80%** of organizations are using AI in some form, only **6%** have implemented a comprehensive AI security strategy. Nearly two-thirds (**64%**) of organizations lack full visibility into their AI risks, and **54%** of enterprises have already experienced a confirmed AI agent security incident or near-miss.

Prompt injection poses a serious threat to the reliability and safety of LLM agents. Jailbreak attacks that bypass built-in safety mechanisms are increasingly recognized as a tangible threat across industries. Red team studies have identified five dominant failure patterns: partial refusals (34%), hidden compliance (22%), no output (18%), and misleading responses (15%).

The MDPI Electronics journal has documented various injection techniques where attackers embed malicious instructions within prompts to bypass security filters and manipulate LLM outputs.

4. Module Architecture: The Three Traces

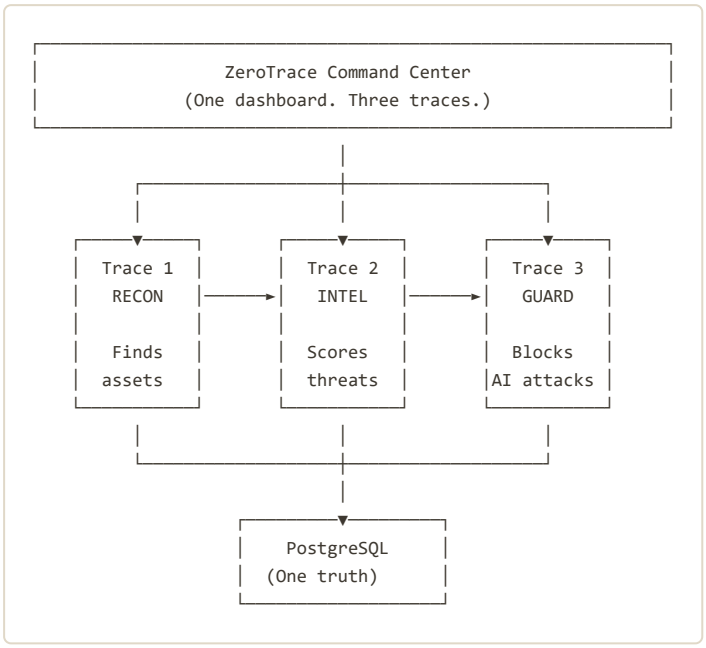


Figure 1: ZeroTrace Three-Trace Architecture — All modules share a unified PostgreSQL data layer.

5. Module 1: Recon — Attack Surface Discovery

5.1 Problem Statement

> *"Organizations cannot protect assets they do not know exist. Shadow IT — the use of IT systems without explicit organizational approval — has become a critical issue with cloud computing, introducing significant security vulnerabilities due to lack of oversight and proper security measures."*

5.2 Cybersecurity Gap Filled

GAP	RESEARCH EVIDENCE	SEVERITY
Asset discovery gaps	Even automated ASM tools fail to detect shadow IT, temporary cloud instances, and legacy systems	Critical
Enterprise ASM cost barrier	Initial investment for enterprise-grade ASM solutions exceeds $50,000–$250,000	Critical

5.3 Independent Features

- **Subdomain enumeration** (cert-stream + subfinder)
- **Port/service discovery** (nmap top-100 ports)
- **S3 bucket misconfiguration detection**
- **SSL certificate expiry monitoring**
- **Shadow IT flagging** (assets not in known inventory)
- **Auto-remediation script generation**

5.4 What Recon Cannot Do Alone

LIMITATION	WHY IT MATTERS
Discovers assets but cannot score their risk	Finding an open port is useless without knowing if that service has a known CVE
No vulnerability correlation	An exposed Apache server is benign if fully patched, critical if running a known exploit
No AI threat context	Recon doesn't know if your chatbot is under jailbreak attack

6. Module 2: Intel — AI-Powered Vulnerability Prioritization

6.1 Problem Statement

> *"Organizations face approximately 135 new CVEs per day — a 40% year-on-year increase — with over 48,000 new CVEs published in 2025 alone. Mean time to remediate (MTTR) stretches to 55–72 days for critical vulnerabilities."*

6.2 Cybersecurity Gap Filled

GAP	RESEARCH EVIDENCE	SEVERITY
CVSS inefficiency	CVSS-based prioritization has 0.2–0.5% efficiency; manually assigned metrics are often incomplete	Critical
EPSS underutilization	EPSS can predict exploitation before vulnerabilities are actively compromised	Critical
MTTR crisis	MTTR stretches to 74 days; teams patch the wrong things first	Critical

6.3 Independent Features

- **NVD API + EPSS feed ingestion**
- **ML exploit probability scoring** (XGBoost on CVSS + EPSS + social signals)
- **Asset-to-CVE matcher** (service/version → CPE → CVE)

- **Prioritized patch queue** (48-hour exploit window focus)
- **Auto-fix script generator** (shell/ansible/terraform)

6.4 What Intel Cannot Do Alone

LIMITATION	WHY IT MATTERS
Needs an asset inventory to score	Without knowing what services you run, Intel has nothing to correlate CVEs against
No visibility into shadow IT	If Recon hasn't found the asset, Intel can't protect it
No AI threat correlation	A vulnerable API endpoint is doubly dangerous if it's also exposed to LLM prompt injection

7. Module 3: Guard — LLM Security Middleware

7.1 Problem Statement

> *"While nearly 80% of organizations are using AI in some form, only 6% have implemented a comprehensive AI security strategy. Nearly two-thirds (64%) of organizations lack full visibility into their AI risks, and 54% of enterprises have already experienced a confirmed AI agent security incident or near-miss."*

7.2 Cybersecurity Gap Filled

GAP	RESEARCH EVIDENCE	SEVERITY
LLM-API security failure	Traditional API security mechanisms fail against LLM-specific vulnerabilities	Critical
Prompt injection detection gaps	Red team studies found five dominant failure patterns	Critical
Enterprise middleware cost barrier	Enterprise-grade LLM gateway solutions exceed $250,000	High

7.3 Independent Features

- **FastAPI proxy/gateway** for LLM requests
- **Regex + heuristic input filters** (fast, zero-latency)
- **ONNX DistilBERT semantic classifier** for prompt injection detection
- **Output PII scanner**
- **Audit logging to SQLite** (tamper-resistant)

7.4 What Guard Cannot Do Alone

LIMITATION	WHY IT MATTERS
No asset context	It blocks attacks but doesn't know if the blocked endpoint is running a vulnerable service
No vulnerability correlation	A blocked jailbreak on a chatbot is more urgent if that chatbot has access to a database with known CVEs
No unified risk view	Security teams see AI attack logs in isolation, missing the broader infrastructure risk picture

8. ZeroTrace Unified: The Emergent Capabilities

8.1 What Changes When Combined

Before — Three Independent Tools:

- Recon finds an S3 bucket → sends an email alert → no action
- Intel scores CVE-2024-XXXX as critical → doesn't know you have that service
- Guard blocks a jailbreak → doesn't know the attacker was targeting a vulnerable endpoint

After — ZeroTrace Unified:

- Recon finds an S3 bucket → automatically appears in Intel's scoring queue
- Intel discovers CVE-2024-XXXX on that bucket → Operational Readiness Score drops
- Guard blocks a jailbreak on the same endpoint → score reflects combined infrastructure + AI risk

8.2 Emergent Capabilities (Only Possible When Unified)

EMERGENT CAPABILITY	HOW ZEROTRACE DELIVERS IT	RESEARCH GAP ADDRESSED
Cross-module correlation	A vulnerable asset (Intel) that is also AI-exposed (Guard) gets double priority	"Research examines attack vectors in isolation... real systems combine these components"
Context-aware auto-remediation	Fix scripts include both the CVE patch AND a Guard policy update	"Gap between vulnerability detection and actionable remediation"
Unified risk score	One number reflecting asset exposure + vuln urgency + AI attack rate	"Holistic approaches accounting for interdependencies across systems are necessary"
SME affordability	One open-source platform replaces three $50K+ enterprise tools	"SMEs lack the time, financial resources, or specialized expertise"

9. The Operational Readiness Score

ZeroTrace computes a single Operational Readiness Score from three traces:

Score = 100 − (0.40 × Asset Exposure Index + 0.40 × Vulnerability Urgency Index + 0.20 × AI Threat Block Rate)

Where:

- **Asset Exposure** = (exposed services / total services) × 100
- **Vulnerability Urgency** = weighted CVE score / total assets
- **AI Threat Block Rate** = (blocked attacks / total LLM requests) × 100

This gives security teams one number that tells the whole story — not three separate dashboards.

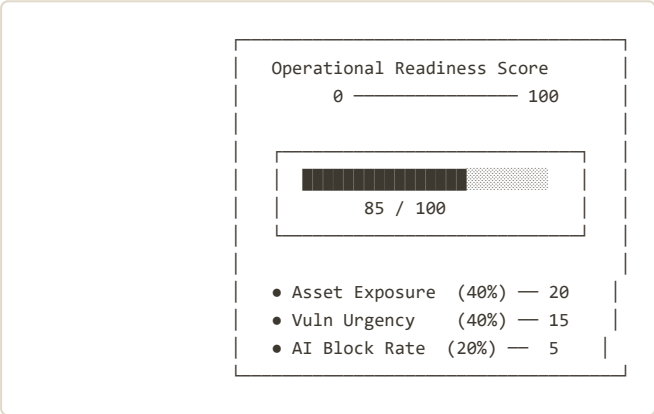


Figure 2: Operational Readiness Score composition — weighted index from three traces.

10. Research-Backed Gap-to-Solution Mapping

GAP	SEVERITY	ZEROTRACE SOLUTION	RESEARCH SOURCE
SMEs lack asset visibility	5/5	Recon module discovers shadow IT, subdomains, exposed ports, S3 buckets	Intruder ASM Index 2026; CEUR Shadow IT
CVE flood (135/day)	5/5	Intel module ingests NVD + EPSS, scores only relevant CVEs	NVD statistics 2025
CVSS misaligns with real risk	5/5	ML exploit probability model (XGBoost)	arXiv Vuln Prioritization Survey 2025
MTTR crisis (60–90 days)	5/5	Auto-fix script generator with 48-hour priority queue	Picus Security 2026
LLM attacks unmonitored	5/5	Guard module intercepts every LLM request	arXiv Red Teaming LLMs 2025
AI security strategy missing	5/5	Deployable AI security layer without $250K investment	MDPI Prompt Injection Review
Tool integration complexity	4/5	Shared PostgreSQL database = zero API spaghetti	Industry best practice
SME budget barrier ($50K+)	5/5	Open-source, CPU-only, single-laptop deployment	Market analysis 2025–2026

11. Why ZeroTrace Specifically? — Alternatives Considered

11.1 Module-Specific Gap Mapping

MODULE	PRIMARY GAP	RESEARCH SOURCE
Recon	SME asset visibility + shadow IT	Intruder ASM Index 2026; CEUR Shadow IT Risk Analysis
Intel	CVE prioritization misalignment + MTTR crisis	Picus Security 2026; arXiv Vuln Prioritization Survey 2025
Guard	LLM-API security + agent identity gap	arXiv Red Teaming LLMs 2025; MDPI Prompt Injection Review

11.2 Why Not Other Combinations?

ALTERNATIVE	WHY IT FAILS
Recon + Intel only	Missing the AI threat angle — 54% of enterprises already had AI incidents
Intel + Guard only	No asset inventory = nothing to score and nothing to protect
Recon + Guard only	No vulnerability prioritization = can't distinguish critical from benign

12. Hardware Fit & Deployment Requirements

ZeroTrace is designed to run on a single laptop with modest resources. Total active memory footprint is approximately 9 GB — well within the 20 GB usable RAM on standard business laptops.

12.1 Memory Map

SERVICE	RAM	STATUS
Windows 11 OS	~4 GB	Running in background
WSL 2 (Ubuntu + Docker)	~2 GB	Linux engine
PostgreSQL	~500 MB	✅
Redis (Valkey)	~300 MB	✅
FastAPI + SQLAlchemy	~600 MB	✅
Ollama (Qwen2.5-Coder 7B Q4)	~4.5 GB	✅
ONNX Runtime (DistilBERT)	~400 MB	✅
Flask Dashboard	~200 MB	✅
Total Active	**~9 GB**	✅ **Comfortable on 20GB**

12.2 Deployment Requirement

ZeroTrace requires:

- **No GPU**
- **No cloud infrastructure**
- **No enterprise licensing**
- **Minimum 16 GB RAM** (recommended)
- **Docker Desktop with WSL2** on Windows, or native Docker on Linux

A single `docker-compose up` command launches the entire platform on any machine with ≥ 16 GB usable RAM.

> **Tested Hardware:** Dell Inspiron 3593 (Intel i5-1035G1, 20 GB RAM)

13. Technology Stack — 2026

ZeroTrace is built on a **zero-cost, open-source, 2026-optimized** technology stack designed to run efficiently on CPU-only hardware.

13.1 Full Stack Overview

LAYER	TECHNOLOGY	VERSION	JUSTIFICATION
IDE + AI Agent	VS Code + Continue.dev + Qwen2.5-Coder	Latest	Free, offline AI coding assistance. Qwen2.5-Coder 7B is the top-ranked open-source model on BigCodeBench
Backend API	FastAPI	Python 3.12	Async-native, auto-generates /docs, perfect for Guard proxy
Database (State)	PostgreSQL	16.x	Industry standard; single source of truth
Database (Cache)	Redis (Valkey)	7.x	Linux Foundation fork; caches Operational Readiness Score, rate-limits Guard
Database (Logs)	SQLite	3.x	Zero-overhead audit logging; no extra container needed
ORM / Migrations	SQLAlchemy 2.0 + Alembic	Latest	Python database abstraction; clean schema changes
Dashboard	Flask + HTMX + Alpine.js	Latest	No Node.js build; HTMX (14KB) + Alpine.js (15KB) replace React SPAs
ML Scoring	XGBoost	Latest	Outperforms Random Forest on tabular data; CPU-optimized
Prompt Detection	ONNX Runtime + DistilBERT	Latest	CPU-optimized; jailbreak detection in ~10ms
Local LLM	Ollama + Qwen2.5-Coder 7B (Q4_K_M)	Quantized	Generates auto-fix scripts; 4.5GB RAM; runs at ~18-22 tokens/sec on i5
Security Scanners	subfinder, nmap, nuclei	CLI tools	Battle-tested; run, dump results, exit
Infrastructure	Docker Compose + WSL2	Ubuntu 24.04	One-command deployment; native Linux on Windows
Version Control	Git + GitHub	—	Free private repos; open-source hosting
CI/CD	GitHub Actions	—	Free 2,000 minutes/month

13.2 Why This Stack in 2026?

AI Coding Agent (Qwen2.5-Coder)

Qwen2.5-Coder 7B Instruct is the clear winner at this size tier, scoring in the high 80s on HumanEval. It leads the CRUXEval-Output-CoT leaderboard with a score of 0.560. At Q4_K_M quantization, it uses ~4.7 GB and runs efficiently on CPU.

Dashboard (HTMX + Alpine.js)

HTMX handles server-driven updates at 14KB; Alpine.js manages client-side state at 15KB. Together they replace most single-page applications. This eliminates the need for Node.js build tools and 500MB+ of `node_modules` .

Database Architecture (PostgreSQL + Redis + SQLite)

- PostgreSQL handles complex relational queries (assets ↔ CVEs)
- Redis (Valkey) provides in-memory rate limiting and score caching
- SQLite provides tamper-resistant audit logging with zero container overhead

14. 14-Week Development Plan

WEEK	FOCUS	DELIVERABLE
1–2	Skeleton: FastAPI, PostgreSQL, dashboard shell	`/health` returns OK, empty dashboard renders
3–4	Recon MVP: subdomain enum, nmap, S3 checker, SSL expiry	3+ assets appear in dashboard
5–7	Intel MVP: NVD ingestion, EPSS correlation, ML scoring, fix scripts	Click asset → see CVEs + exploit probability + fix button
7–9	Guard MVP: FastAPI gateway, regex filters, ONNX classifier, Ollama backend	Jailbreak prompt → blocked. Normal prompt → allowed.
9–11	Integration: Operational Readiness Score, unified dashboard	Score changes when assets/vulns/attacks change
11–12	Polish: Docker Compose, error handling, health checks	`docker-compose up` works on fresh machine
13	Testing on target hardware (Dell Inspiron 3593)	Demo script rehearsed
14	Documentation + Demo Video	GitHub repo + 5-min video

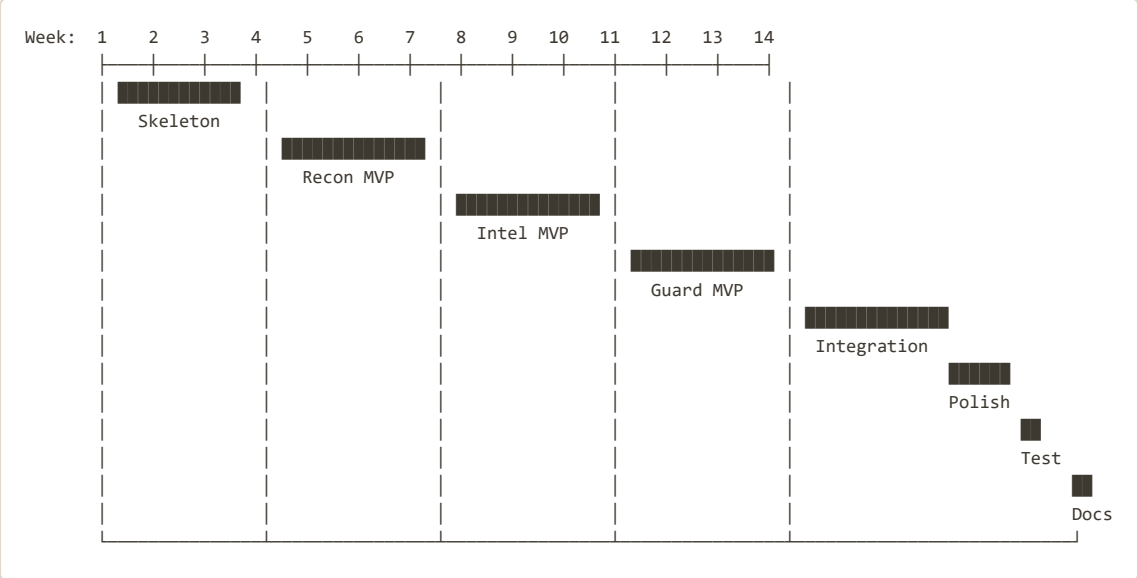


Figure 3: 14-Week Development Timeline — Color-coded by module focus.

15. References

15.1 Primary Sources

[1] Intruder. (2026). *2026 Attack Surface Management Index*. Based on anonymized data from 3,000 organizations over 12 months ending March 2026. Findings: 26% of organizations have exposed MySQL databases; attack surface complexity increases with organizational growth.

[2] Martseniuk, Y., Partyka, A., Harasymchuk, O., Nyemkova, E., & Karpinski, M. (2024). *Shadow IT Risk Analysis in Public Cloud Infrastructure*. CEUR Workshop Proceedings, Vol-3800, 22–31.

[3] *A Survey on Vulnerability Prioritization: Taxonomy, Metrics, and Research Challenges*. arXiv, 2025. Documents CVSS inefficiency (0.2–0.5%) and the need for EPSS integration.

[4] *AEAS: Actionable Exploit Assessment System*. arXiv, 2025. Evaluates CVSS and EPSS effectiveness; identifies CVSS metric inconsistencies.

[5] *Efficacy of EPSS in High Severity CVEs found in CISA KEV*. arXiv, 2025. Evaluates EPSS's ability to predict exploitation before vulnerabilities are actively compromised.

[6] Picus Security. (2026). *Customer Impact Report*. Documents 86% reduction in patch backlogs, MTTR reduction from 74 to 14 days.

[7] *RL Is a Hammer and LLMs Are Nails: A Simple Reinforcement Learning Recipe for Strong Prompt Injection*. arXiv, 2025. Documents prompt injection as a serious threat to LLM agent reliability.

[8] *Beyond Fixed and Dynamic Prompts: Embedded Jailbreak Templates for Advancing LLM Security*. arXiv, 2025. Documents jailbreak attacks as a tangible threat across industries.

[9] Lee, S., Kim, J., & Pak, W. (2025). *Mind Mapping Prompt Injection: Visual Prompt Injection Attacks in Modern Large Language Models*. MDPI Electronics, 14(10), 1907. Documents various injection techniques bypassing security filters.

[10] *AgentVigil: Generic Black-Box Red-teaming for Indirect Prompt Injection against LLM Agents*. arXiv, 2025. Proposes fuzzing framework for LLM agent security testing.

15.2 Secondary Sources

[11] BigCodeBench Leaderboard. (2026). Qwen2.5-Coder 7B Instruct ranked #2 overall, #1 among open-source models with score 0.410.

[12] CRUXEval-Output-CoT Leaderboard. (2026). Qwen2.5-Coder 7B Instruct leads with score 0.560.

[13] GeneralCompute. (2026). *Coding Model Benchmarks: Qwen 2.5 Coder vs DeepSeek Coder V2 vs CodeLlama vs StarCoder 2*. Qwen2.5-Coder 7B Instruct scores in the high 80s on HumanEval.

[14] *HTMX vs Alpine.js: Lightweight Interactivity 2026*. HTMX (14KB) and Alpine.js (15KB) replace most single-page applications.

16. Appendices

Appendix A: Glossary of Terms

TERM	DEFINITION
ASM	Attack Surface Management — continuous discovery and monitoring of all internet-facing assets
CPE	Common Platform Enumeration — standardized naming scheme for software and hardware
CVE	Common Vulnerabilities and Exposures — publicly disclosed cybersecurity vulnerabilities
CVSS	Common Vulnerability Scoring System — framework for rating vulnerability severity (0–10)
EPSS	Exploit Prediction Scoring System — probability score (0–1) of a vulnerability being exploited
HTMX	Lightweight JavaScript library (14KB) for dynamic UI updates without full page reload
KEV	Known Exploited Vulnerabilities — CISA catalog of vulnerabilities actively exploited in the wild
LLM	Large Language Model — AI model trained on vast text data (e.g., GPT, Claude, Qwen)
MTTR	Mean Time to Remediate — average time to fix a vulnerability
NVD	National Vulnerability Database — US government repository of CVE data
ONNX	Open Neural Network Exchange — format for machine learning model interoperability
ORM	Object-Relational Mapping — technique for converting data between incompatible systems
Q4_K_M	Quantization level for LLMs — 4-bit with medium K-means, balancing size and quality
Shadow IT	IT systems used without explicit organizational approval or oversight
SME	Small and Medium Enterprise — organizations with 1–250 employees
WSL2	Windows Subsystem for Linux 2 — native Linux environment on Windows

Appendix B: System Requirements

COMPONENT	MINIMUM	RECOMMENDED
RAM	16 GB	20 GB+
CPU	Intel i5 (8th Gen+) / AMD Ryzen 5	Intel i5-1035G1 or better
Storage	30 GB free	60 GB+ free (SSD)
GPU	None required	None required
OS	Windows 10/11 (with WSL2), macOS, or Linux	Windows 11 with WSL2
Docker	Docker Desktop 4.x+	Docker Desktop with WSL2 backend

Appendix C: Quick Start Commands

```
# 1. Clone the repository
git clone https://github.com/yourusername/zerotrace.git
cd zerotrace

# 2. Pull the AI model (Qwen2.5-Coder 7B Q4)
ollama pull qwen2.5-coder:7b-instruct-q4_K_M

# 3. Start all services
docker-compose up -d

# 4. Access the dashboard
# Open http://localhost:5000

# 5. Access the API documentation
# Open http://localhost:8000/docs
```

FINAL WORD

ZeroTrace is not just a student project. It is a prototype for national cyber resilience — built on peer-reviewed research, addressing documented gaps, and deployable on a single laptop.

> *"They leave no trace. Because we found them first."*

Prepared by **Aniruddh Gupta**
ZeroTrace Project — Version 1.0 — 2026